Executive Summary

High-Concurrency Hotel Reservation System — Phase 1 Infrastructure Design

---

1. Business Context

A hotel chain operating **5,000 properties** with approximately **1 million rooms** requires a reservation system capable of handling extreme concurrency during peak booking periods — flash sales, holiday seasons, and event-driven demand spikes — while guaranteeing that **no two customers can book the same room for the same date**.

The core challenge is not just scale, but **correctness under contention**. A single popular hotel during a major event can receive thousands of simultaneous booking attempts for a handful of remaining rooms. The system must resolve these races with zero double-bookings while keeping response times acceptable.

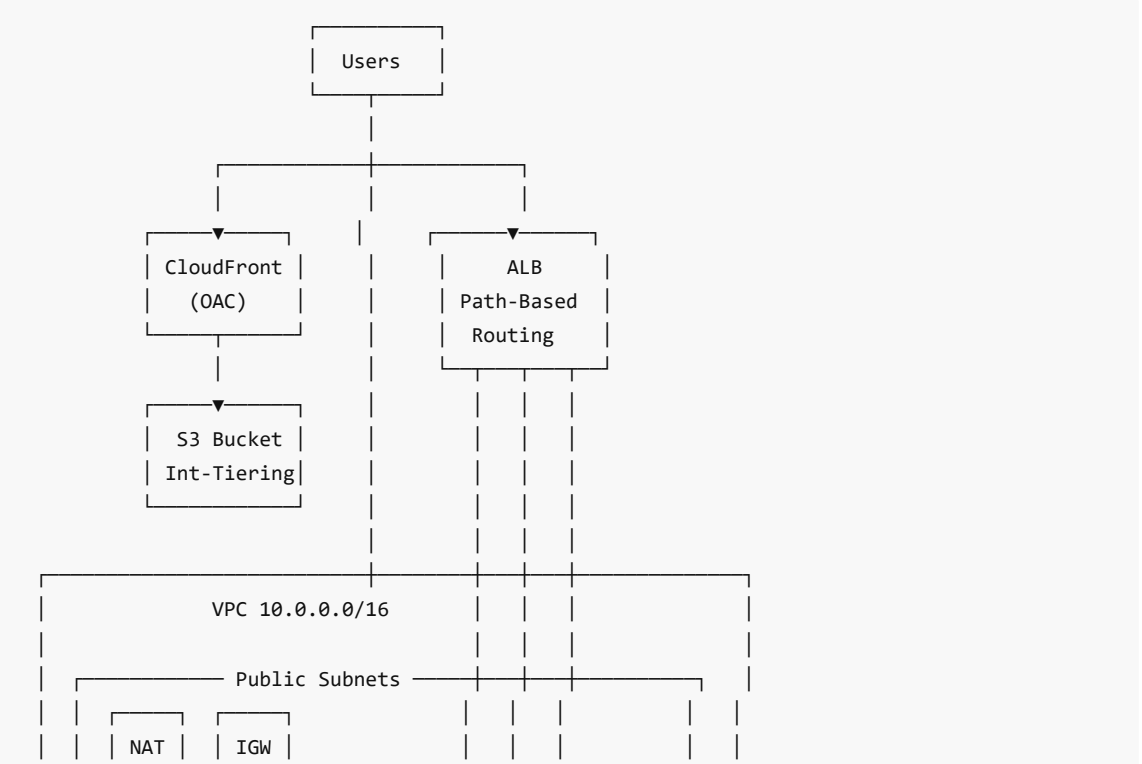
Key Business Requirements:

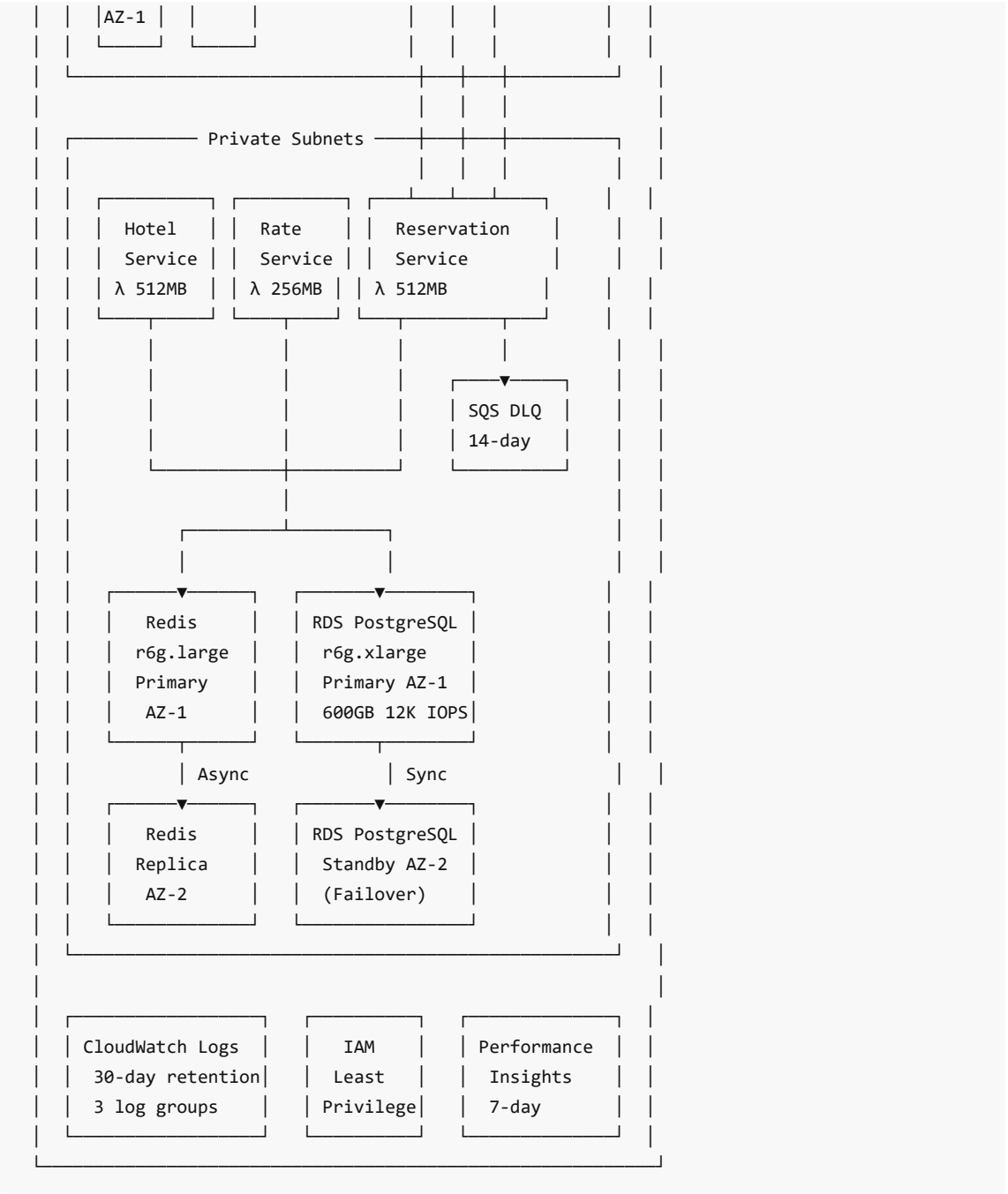
- **Zero double-bookings** — ACID transactions with database-level enforcement
- **10% overbooking support** — configurable buffer to account for anticipated cancellations
- **Sub-second browsing latency** — fast availability checks for search and browse flows
- **Acceptable reservation latency** — a few seconds is tolerable for the booking transaction itself
- **Idempotent APIs** — safe retries using reservation-level idempotency keys

---

2. Architecture Overview

The system follows a **serverless microservices** pattern on AWS, with three independently scalable services behind an Application Load Balancer. All compute and data resources reside in private subnets, with no direct internet exposure.





---

3. High Availability & Resilience

The system is designed to survive the failure of an entire Availability Zone without data loss or service interruption.

Multi-AZ Data Tier:

Component	AZ-1 (Primary)	AZ-2 (Standby/Replica)
RDS PostgreSQL 16.4	Primary — handles all writes	Standby — synchronous replication
ElastiCache Redis 7.1	Primary — handles reads/writes	Replica — async replication, failover

Failure Handling:

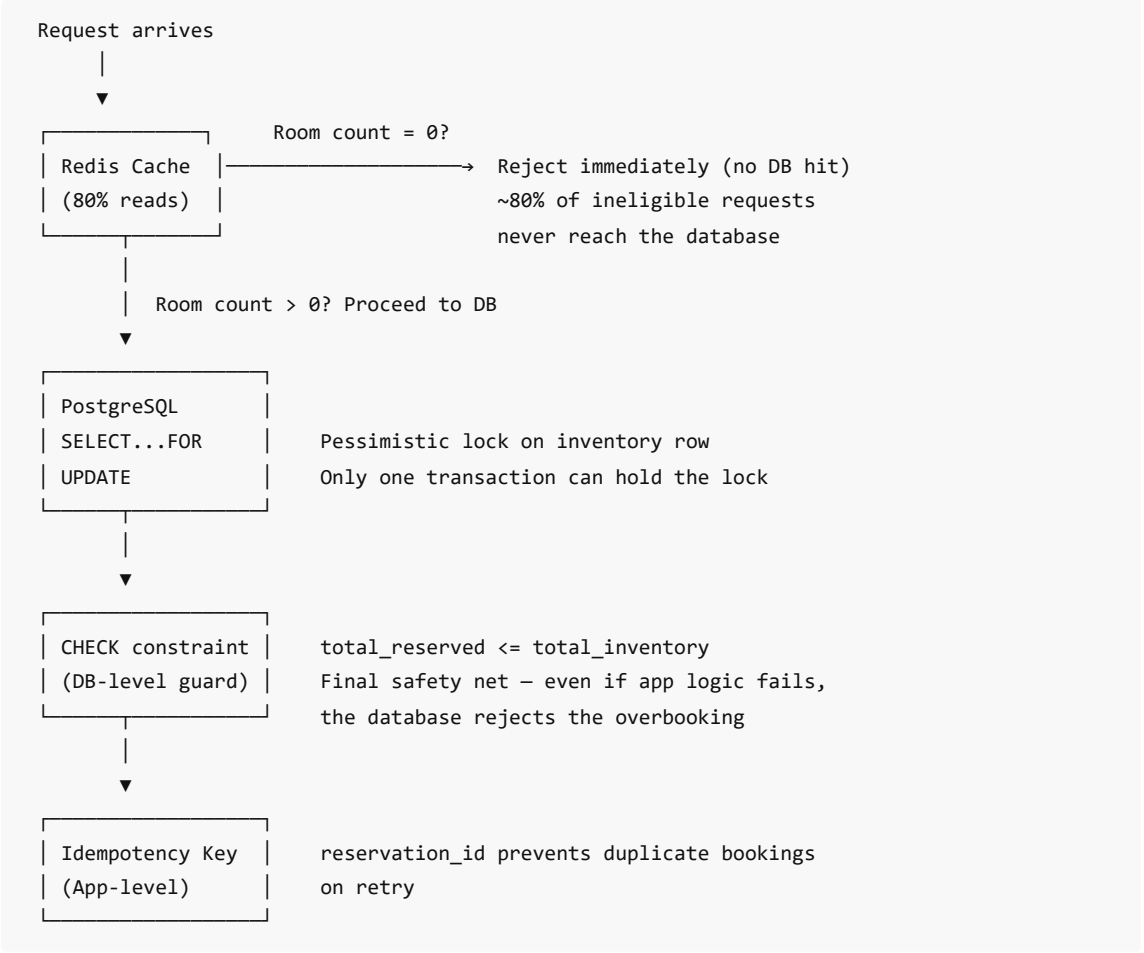
Failure Scenario	System Response
AZ-1 outage	RDS auto-failover to AZ-2 standby, Redis promotes replica
Lambda invocation failure	SQS DLQ captures event for retry/investigation
S3 origin unavailable	CloudFront serves cached content from edge locations
Database connection exhaustion	Redis cache absorbs 80%+ of read traffic

---

4. Concurrency & Data Consistency

This is the hardest problem in the system. When thousands of users attempt to book the last available room simultaneously, the system must guarantee exactly one succeeds.

Strategy: Defense in Depth



Data Model:

Reservations are made at the **room type** level (e.g., "King" or "Standard"), not for a specific room number. Room assignment happens at check-in. Inventory is tracked in a `room_type_inventory` table with a composite primary key of `(hotel_id, room_type_id, date)`, covering a rolling two-year window (~730 million rows at full scale).

---

5. Performance at Scale

Compute Tier

Service	Memory	Reserved Concurrency	ALB Route	Role
Hotel Service	512 MB	100	/hotels	CRUD for hotels and rooms, images
Rate Service	256 MB	50	/rates	Dynamic pricing lookups
Reservation Service	512 MB	200	/reservations	Booking creation and cancellation

The Reservation Service has the highest concurrency allocation because it handles the most contention-sensitive workload. The Rate Service uses less memory since pricing lookups are lightweight read operations.

Data Tier

Resource	Instance	Storage	Purpose
RDS PostgreSQL	db.r6g.xlarge	600 GB gp3, 12K IOPS	Transactional store, ACID guarantees
ElastiCache Redis	cache.r6g.large	In-memory	Availability cache, reduces DB reads by 80%

The `r6g` (Graviton2) instance family provides the best price-performance ratio for memory-intensive workloads. The 12K provisioned IOPS on gp3 storage ensures consistent write performance during peak booking periods.

---

6. Security & Network Isolation

The architecture follows a **zero-trust network model** where every layer only accepts traffic from the layer directly above it.

Layer	Subnet	Accepts Traffic From	Ports
ALB	Public	Internet (0.0.0.0/0)	80, 443
Lambda Functions	Private	ALB only (VPC internal)	N/A
RDS PostgreSQL	Private	Lambda Security Group only	5432
ElastiCache Redis	Private	Lambda Security Group only	6379
S3 Bucket	N/A	CloudFront OAC only	443

IAM Least Privilege: Each Lambda function has its own IAM role with permissions scoped to exactly what it needs:

- **Hotel Service** → CloudWatch Logs + S3 (read/write hotel images)
- **Rate Service** → CloudWatch Logs only
- **Reservation Service** → CloudWatch Logs + SQS (send to DLQ)

All three services access RDS and Redis via VPC networking (security groups), not IAM — the database credentials are passed as environment variables.

---

7. Operational Efficiency & Cost Optimization

Strategy	Implementation	Impact
Serverless Compute	Lambda scales to zero during off-peak hours	Pay only for actual booking requests
Caching Layer	Redis absorbs 80%+ of read traffic	Reduces RDS costs significantly
Storage Tiering	S3 Intelligent-Tiering (Archive 90d, Deep Archive 180d)	Automatic cost reduction for old images
Single NAT Gateway	One NAT in AZ-1 instead of per-AZ	~50% NAT cost savings
Graviton2 Instances	r6g family for RDS and Redis	~20% better price-performance vs x86
CloudFront PriceClass_100	Edge locations in lowest-cost regions	Reduced CDN costs

Observability:

- CloudWatch Log Groups per Lambda function with 30-day retention
- RDS Performance Insights enabled (7-day retention) for query-level monitoring
- RDS automated backups with 7-day retention window

---

8. Design Decisions & Trade-offs

Decision	Rationale	Trade-off
ALB instead of API Gateway	Lower cost at high throughput, VPC-native Lambda integration	No built-in API key management or throttling
Single NAT Gateway (AZ-1 only)	Cost savings (~$32/month saved)	If AZ-1 fails, Lambda loses internet access
Pessimistic locking over optimistic	Guarantees correctness under extreme contention	Higher latency per reservation transaction
Room-type reservations (not specific)	Simplifies inventory management, reduces lock contention	Room assignment deferred to check-in
Redis as read cache (not write-through)	Database remains single source of truth	Cache can be briefly stale after writes
gp3 storage over io2	Sufficient IOPS at lower cost for current scale	May need io2 if write patterns intensify

---

9. Technology Stack

Component	Technology

Cloud Provider	AWS
IaC / Deployment	Kiro (CloudFormation)
Load Balancing	Application Load Balancer (ALB)
Compute	AWS Lambda (Node.js 20.x)
Database	RDS PostgreSQL 16.4 (db.r6g.xlarge, 600GB, Multi-AZ)
Caching	ElastiCache Redis 7.1 (cache.r6g.large, Multi-AZ)
Static Content	S3 (Intelligent-Tiering) + CloudFront CDN (OAC)
Messaging	SQS (Dead Letter Queue)
Observability	CloudWatch Logs + RDS Performance Insights
Security	IAM (least privilege), VPC, Security Groups

---

10. Infrastructure Resources

Resource	Specification
VPC	10.0.0.0/16 — 2 AZs, 4 subnets
Public Subnets	10.0.1.0/24 (AZ-1), 10.0.2.0/24 (AZ-2)
Private Subnets	10.0.10.0/24 (AZ-1), 10.0.11.0/24 (AZ-2)
ALB	Internet-facing, spans both public subnets
NAT Gateway	Single, AZ-1 (cost-optimized)
Hotel Service Lambda	512 MB, 100 reserved concurrency
Rate Service Lambda	256 MB, 50 reserved concurrency
Reservation Service Lambda	512 MB, 200 reserved concurrency + DLQ
RDS PostgreSQL	db.r6g.xlarge, 600GB gp3, 12K IOPS, Multi-AZ
ElastiCache Redis	cache.r6g.large, 1 replica, Multi-AZ
S3 Bucket	Intelligent-Tiering (Archive 90d, Deep 180d)
CloudFront	PriceClass_100, HTTP/2+3, OAC
SQS DLQ	14-day message retention
CloudWatch Log Groups	3 groups, 30-day retention

---

11. Next Steps — Phase 2

Initiative	Description

HTTPS on ALB	Add ACM certificate and HTTPS listener (port 443)
Authentication	Integrate Amazon Cognito for user auth and API authorization
Database Sharding	Shard by `hotel_id` when single-instance limits are reached
Second NAT Gateway	Add NAT in AZ-2 for full AZ-independent resilience
CI/CD Pipeline	Automated testing and deployment via CodePipeline
Load Testing	Validate concurrency limits with realistic booking scenarios
Monitoring & Alerting	CloudWatch Alarms for Lambda errors, RDS CPU, Redis evictions
WAF Integration	AWS WAF on ALB for DDoS protection and rate limiting

---

Phase 1 — Hotel Reservation System Infrastructure Design Generated with Kiro